

Capítulo 2: Introdução à Modelagem Matemática

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

23 de maio de 2026

2.1 Introdução à Modelagem Matemática

Objetivos de Aprendizagem

- Compreender o que são modelos matemáticos
- Aprender a construir modelos para sistemas biológicos
- Dominar técnicas de análise de modelos
- Aplicar modelagem a problemas reais

Definição: Modelo Matemático

Representação abstrata de um sistema real usando linguagem matemática, que permite fazer previsões quantitativas sobre o comportamento do sistema.

Por que Modelar Sistemas Biológicos?

Vantagens da Modelagem:

- Previsão de comportamento
- Teste de hipóteses *in silico*
- Otimização de experimentos
- Identificação de componentes chave
- Design racional de sistemas

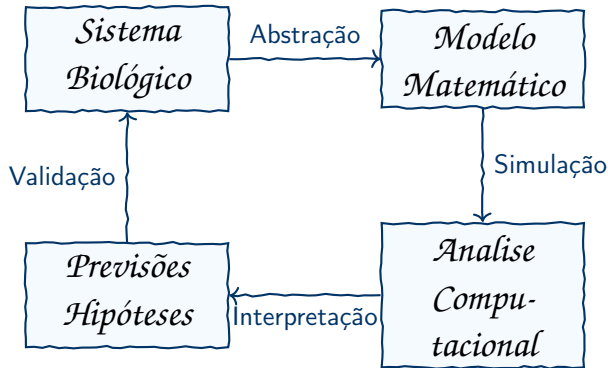
Aplicações:

- Dinâmica de populações
- Vias metabólicas
- Redes regulatórias
- Farmacocinética
- Biologia sintética
- Medicina personalizada

Importante!

"All models are wrong, but some are useful— George Box

O Ciclo da Modelagem



Ciclo Iterativo

O processo é iterativo: modelos são refinados com base em novos dados experimentais

Classificação de Modelos

Por Natureza Matemática

- **Determinísticos:** resultado único para cada conjunto de condições
- **Estocásticos:** incorporam aleatoriedade e probabilidade

Por Dependência Temporal

- **Estáticos:** não mudam com o tempo (redes, equilíbrio)
- **Dinâmicos:** evolução temporal (EDOs, EDPs)

Por Escala

- **Microscópicos:** nível molecular/celular
- **Mesoscópicos:** populações celulares
- **Macroscópicos:** tecidos, órgãos, organismos

Modelos Contínuos vs Discretos

Modelos Contínuos

- Variáveis contínuas
- Equações diferenciais
- Concentrações
- Grande número de moléculas

Exemplo:

$$\frac{d[X]}{dt} = k_1 - k_2[X]$$

Modelos Discretos

- Contagem de moléculas
- Algoritmos estocásticos
- Eventos discretos
- Pequeno número de moléculas

Exemplo:

$$X(t + 1) = X(t) + \text{prod} - \text{deg}$$

Quando usar cada tipo?

- Contínuo: >100 moléculas
- Discreto: <100 moléculas

Modelos Empíricos vs Mecanísticos

Empíricos

Baseados em dados

- Ajuste de curvas
- Regressão estatística
- Machine learning
- Pouca interpretação biológica

Vantagens:

- Simples e rápidos
- Bom ajuste aos dados

Desvantagens:

- Pouco poder preditivo
- Extrapolação arriscada

Mecanísticos

Baseados em mecanismos

- Representam processos reais
- Leis de conservação
- Princípios físico-químicos
- Interpretação biológica

Vantagens:

- Poder preditivo
- Extrapolação confiável

Desvantagens:

- Complexos
- Muitos parâmetros

Etapas da Construção de Modelos

1. Definição do problema

- Objetivo claro
- Escopo bem definido
- Questões a responder

2. Identificação de variáveis

- Variáveis de estado
- Parâmetros
- Entradas e saídas

3. Formulação de equações

- Leis de conservação
- Cinéticas de reação
- Condições iniciais e de contorno

4. Analise e simulação

Exemplo: Modelo de Decaimento Radioativo

Sistema

Um isótopo radioativo decai espontaneamente ao longo do tempo

1. **Variável de estado:** $N(t)$ = número de núcleos no tempo t
2. **Hipótese:** Taxa de decaimento proporcional ao número de núcleos
3. **Equação:**

$$\frac{dN}{dt} = -\lambda N$$

4. **Solução analítica:**

$$N(t) = N_0 e^{-\lambda t}$$

onde $N_0 = N(0)$ é a condição inicial e λ é a constante de decaimento.

Implementação: Decaimento Radioativo I

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 # Definir a EDO
6 def decay(N, t, lam):
7     """Taxa de variacao de N"""
8     return -lam * N
9
10 # Parametros
11 lambda_val = 0.5 # constante de decaimento
12 N0 = 100         # quantidade inicial
13 t = np.linspace(0, 10, 100)
```

Listing 1: Solução numérica do decaimento

Implementação: Decaimento Radioativo (2/2)

```
1 # Resolver numericamente
2 N_num = odeint(decay, N0, t, args=(lambda_val,))
3
4 # Solucao analitica
5 N_ana = N0 * np.exp(-lambda_val * t)
6
7 # Plotar
8 plt.plot(t, N_num, 'b-', label='Numerica')
9 plt.plot(t, N_ana, 'r--', label='Analitica')
10 plt.xlabel('Tempo')
11 plt.ylabel('N(t)')
12 plt.legend()
```

Listing 2: Solução e Visualização

Equações Diferenciais Ordinárias (EDOs)

Definição: EDO

Equação que relaciona uma função com suas derivadas em relação a uma variável independente (geralmente o tempo).

Forma Geral

$$\frac{dx}{dt} = f(x, t, p)$$

onde:

- x = vetor de variáveis de estado
- t = tempo
- p = vetor de parâmetros
- f = função que define a dinâmica

Classificação de EDOs

Por Ordem

- **Primeira ordem:** $\frac{dx}{dt} = f(x, t)$
- **Segunda ordem:** $\frac{d^2x}{dt^2} = f(x, \frac{dx}{dt}, t)$
- **Ordem n :** derivadas até n -ésima ordem

Por Linearidade

- **Linear:** $\frac{dx}{dt} = ax + b$
- **Não-linear:** $\frac{dx}{dt} = ax^2 + bx$

Por Autonomia

- **Autônoma:** $\frac{dx}{dt} = f(x)$ (não depende explicitamente de t)
- **Não-autônoma:** $\frac{dx}{dt} = f(x, t)$

Exemplo: Crescimento Exponencial

Modelo Simples de Crescimento

Uma população cresce proporcionalmente ao seu tamanho:

$$\frac{dN}{dt} = rN$$

onde r é a taxa de crescimento per capita.

Solução analítica:

$$N(t) = N_0 e^{rt}$$

Se $r > 0$: crescimento

Se $r < 0$: decaimento

Se $r = 0$: estacionário

Tempo de Duplicação

$$t_2 = \frac{\ln 2}{r}$$

Exemplo: Crescimento Logístico

Modelo de Verhulst

População com capacidade de suporte limitada K :

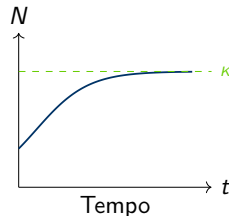
$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right)$$

Solução analítica:

$$N(t) = \frac{KN_0 e^{rt}}{K + N_0(e^{rt} - 1)}$$

Comportamento:

- $N \rightarrow K$ quando $t \rightarrow \infty$
- Forma sigmoidal
- Ponto de inflexão em $N = K/2$



Implementação: Crescimento Logístico I

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def logistic(N, t, r, K):
6     """Modelo de crescimento logístico"""
7     return r * N * (1 - N/K)
8
9 # Parametros
10 r = 0.5      # taxa de crescimento
11 K = 100      # capacidade de suporte
12 N0 = 10      # populacao inicial
13
14 # Tempo
15 t = np.linspace(0, 20, 200)
```

Implementação: Crescimento Logístico II

```
16  
17 # Resolver  
18 N = odeint(logistic, N0, t, args=(r, K))
```

Listing 3: Modelo logístico

Implementação: Crescimento Logístico (2/2)

```
1 # Plotar
2 plt.figure(figsize=(10, 6))
3 plt.plot(t, N, 'b-', linewidth=2, label='N(t)')
4 plt.axhline(y=K, color='r', linestyle='--',
5             label=f'K = {K}')
6 plt.xlabel('Tempo')
7 plt.ylabel('Populacao N(t)')
8 plt.legend()
9 plt.grid(True)
```

Listing 4: Visualização

Sistemas Acoplados

Sistema Geral

Quando múltiplas variáveis interagem:

$$\begin{aligned}\frac{dx_1}{dt} &= f_1(x_1, x_2, \dots, x_n, t) \\ \frac{dx_2}{dt} &= f_2(x_1, x_2, \dots, x_n, t) \\ &\vdots \\ \frac{dx_n}{dt} &= f_n(x_1, x_2, \dots, x_n, t)\end{aligned}$$

Notação vetorial:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

Exemplo: Modelo Predador-Presa (Lotka-Volterra)

Sistema

$$\begin{aligned}\frac{dV}{dt} &= \alpha V - \beta VP && \text{(Presas)} \\ \frac{dP}{dt} &= \delta \beta VP - \gamma P && \text{(Predadores)}\end{aligned}$$

onde:

- V = população de presas
- P = população de predadores
- α = taxa de crescimento das presas
- β = taxa de predação
- γ = taxa de morte dos predadores
- δ = eficiência de conversão

Interpretação do Modelo Predador-Presa

Equação das Presas:

$$\frac{dV}{dt} = \underbrace{\alpha V}_{\text{crescimento}} - \underbrace{\beta VP}_{\text{predação}}$$

- Crescem exponencialmente sem predadores
- Perdem indivíduos por predação
- Taxa de predação $\propto$ encontros (VP)

Equação dos Predadores:

$$\frac{dP}{dt} = \underbrace{\delta \beta VP}_{\text{crescimento}} - \underbrace{\gamma P}_{\text{morte}}$$

- Crescem ao consumir presas
- Morrem naturalmente
- Dependem das presas para sobreviver

Importante!

Este sistema exibe oscilações periódicas características!

Implementação: Lotka-Volterra I

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def lotka_volterra(y, t, alpha, beta, delta, gamma):
6     """Sistema Lotka-Volterra"""
7     V, P = y
8     dVdt = alpha*V - beta*V*P
9     dPdt = delta*beta*V*P - gamma*P
10    return [dVdt, dPdt]
11
12 # Parametros
13 alpha, beta, delta, gamma = 1.0, 0.1, 0.75, 1.5
14 y0, t = [10, 5], np.linspace(0, 50, 1000)
```

Listing 5: Sistema predador-presa

Implementação: Lotka-Volterra (2/2)

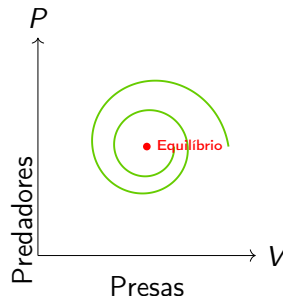
```
1 # Resolver
2 sol = odeint(lotka_volterra, y0, t,
3             args=(alpha, beta, delta, gamma))
4
5 # Plotar series temporais
6 plt.plot(t, sol[:, 0], 'b-', label='Presas')
7 plt.plot(t, sol[:, 1], 'r-', label='Predadores')
8 plt.xlabel('Tempo')
9 plt.ylabel('Populacao')
10 plt.legend()
```

Listing 6: Resolução e Plotting

Retrato de Fase

```
1 # Retrato de fase
2 plt.figure()
3 plt.plot(sol[:, 0], sol[:, 1],
4          'g-', linewidth=2)
5 plt.plot(sol[0, 0], sol[0, 1],
6          'go', markersize=10,
7          label='Inicio')
8 plt.xlabel('Presas (V)')
9 plt.ylabel('Predadores (P)')
10 plt.title('Retrato de Fase')
11 plt.legend()
12 plt.grid(True)
13
14 # Campo vetorial
15 V = np.linspace(0, 20, 20)
16 P = np.linspace(0, 15, 20)
17 V_grid, P_grid = np.meshgrid(V, P)
18 dV, dP = lotka_volterra(
19     [V_grid, P_grid], 0,
20     alpha, beta, delta, gamma)
21 plt.quiver(V_grid, P_grid,
22            dV, dP, alpha=0.5)
```

Listing 7: Plano de fase



Características:

- Trajetórias fechadas
- Oscilações periódicas
- Centro estável

Pontos de Equilíbrio (Steady States)

Definição: Ponto de Equilíbrio

Estado no qual o sistema não muda com o tempo:

$$\frac{d\mathbf{x}}{dt} = 0 \quad \Rightarrow \quad \mathbf{f}(\mathbf{x}^*, t) = 0$$

Exemplo: Crescimento Logístico

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right) = 0$$

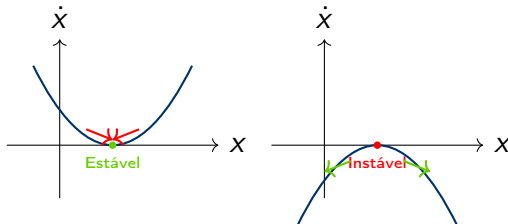
Pontos de equilíbrio:

- $N_1^* = 0$ (extinção)
- $N_2^* = K$ (capacidade de suporte)

Estabilidade de Pontos de Equilíbrio

Classificação

- **Estável:** sistema retorna ao equilíbrio após perturbação
- **Instável:** sistema se afasta após perturbação
- **Marginalmente estável:** comportamento intermediário



Análise de Estabilidade Linear

Linearização em Torno do Equilíbrio

Para $\mathbf{x} = \mathbf{x}^* + \delta\mathbf{x}$ (pequena perturbação):

$$\frac{d\delta\mathbf{x}}{dt} = \mathbf{J}(\mathbf{x}^*)\delta\mathbf{x}$$

onde $\mathbf{J}$ é a matriz Jacobiana:

$$J_{ij} = \left. \frac{\partial f_i}{\partial x_j} \right|_{\mathbf{x}^*}$$

Critério de Estabilidade

O equilíbrio $\mathbf{x}^*$ é estável se todos os autovalores de $\mathbf{J}(\mathbf{x}^*)$ têm parte real negativa.

Exemplo: Análise do Crescimento Logístico

Modelo

$$\frac{dN}{dt} = f(N) = rN \left(1 - \frac{N}{K}\right)$$

Jacobiano (1D):

$$J = \frac{df}{dN} = r \left(1 - \frac{2N}{K}\right)$$

Nos pontos de equilíbrio:

- $N^* = 0$: $J(0) = r > 0 \Rightarrow$ instável
- $N^* = K$: $J(K) = -r < 0 \Rightarrow$ estável

Importante!

A população converge para a capacidade de suporte K

Analise de Estabilidade Computacional I

```
1 import numpy as np
2 from scipy.linalg import eig
3
4 def jacobian_lotka_volterra(y, alpha, beta, delta, gamma):
5     V, P = y
6     return np.array([
7         [alpha - beta*P, -beta*V],
8         [delta*beta*P, delta*beta*V - gamma]
9     ])
10
11 # Ponto de equilibrio e Jacobiano
12 V_eq, P_eq = gamma / (delta * beta), alpha / beta
13 J = jacobian_lotka_volterra([V_eq, P_eq], alpha, beta, delta,
14     gamma)
```

```
15 # Autovalores
16 eigenvalues, _ = eig(J)
17 print(f"Autovalores: {eigenvalues}")
```

Listing 8: Cálculo do Jacobiano e autovalores

Análise de Estabilidade Computacional

```
1 # Ponto de equilibrio
2 V_eq = gamma / (delta * beta)
3 P_eq = alpha / beta
4
5 # Calcular Jacobiano no equilibrio
6 J = jacobian_lotka_volterra([V_eq, P_eq],
7                               alpha, beta, delta, gamma)
8
9 # Autovalores
10 eigenvalues, eigenvectors = eig(J)
11 print(f"Autovalores: {eigenvalues}")
12 print(f"Partes reais: {eigenvalues.real}")
```

Listing 9: Análise de autovalores

Por que Métodos Numéricos?

Limitações das Soluções Analíticas

- Maioria dos sistemas biológicos são não-lineares
- Poucos modelos têm solução analítica fechada
- Sistemas complexos requerem aproximação numérica

Métodos Numéricos Comuns

- **Euler:** simples, primeira ordem
- **Runge-Kutta (RK4):** preciso, quarta ordem
- **Métodos adaptativos:** passo variável (RK45, Dormand-Prince)
- **Métodos implícitos:** sistemas rígidos (BDF)

Método de Euler I

Ideia Básica

Aproximar derivada por diferença finita:

$$\frac{dx}{dt} \approx \frac{x(t+h) - x(t)}{h}$$

$$x(t+h) = x(t) + h \cdot f(x(t), t)$$

Algoritmo:

1. Partir de x_0 em t_0
2. Calcular $x_1 = x_0 + h \cdot f(x_0, t_0)$

3. Repetir: $x_{n+1} = x_n + h \cdot f(x_n, t_n)$

Erro

Erro local: $O(h^2)$, Erro global: $O(h)$

Implementação do Método de Euler I

```
1 def euler(f, x0, t):
2     n = len(t)
3     x = np.zeros((n, len(x0)))
4     x[0] = x0
5     for i in range(n-1):
6         h = t[i+1] - t[i]
7         x[i+1] = x[i] + h * np.array(f(x[i], t[i]))
8     return x
9
10 # Exemplo
11 def exp_growth(x, t, r=0.5): return r * x
12 t = np.linspace(0, 10, 100)
13 x_euler = euler(lambda x,t: exp_growth(x,t), [1.0], t)
```

Listing 10: Euler forward

Método de Runge-Kutta de 4ª Ordem (RK4) I

Ideia

Usar múltiplas avaliações da derivada para melhor aproximação

$$k_1 = f(x_n, t_n)$$

$$k_2 = f\left(x_n + \frac{h}{2}k_1, t_n + \frac{h}{2}\right)$$

$$k_3 = f\left(x_n + \frac{h}{2}k_2, t_n + \frac{h}{2}\right)$$

$$k_4 = f(x_n + hk_3, t_n + h)$$

$$x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Método de Runge-Kutta de 4ª Ordem (RK4) II

Precisão

Erro local: $O(h^5)$, Erro global: $O(h^4)$

Implementação do RK4 I

```
1 def rk4(f, x0, t):
2     n, x = len(t), np.zeros((len(t), len(x0)))
3     x[0] = x0
4     for i in range(n-1):
5         h = t[i+1] - t[i]
6         k1 = np.array(f(x[i], t[i]))
7         k2 = np.array(f(x[i] + h*k1/2, t[i] + h/2))
8         k3 = np.array(f(x[i] + h*k2/2, t[i] + h/2))
9         k4 = np.array(f(x[i] + h*k3, t[i] + h))
10        x[i+1] = x[i] + (h/6) * (k1 + 2*k2 + 2*k3 + k4)
11    return x
12
13 # Usar
14 x_rk4 = rk4(lambda x,t: exp_growth(x,t), [1.0], t)
```

Listing 11: Runge-Kutta 4

Definição: Sensibilidade

Medida de como a saída de um modelo varia em resposta a mudanças nos parâmetros de entrada.

Por que é importante?

- Identificar parâmetros críticos
- Priorizar experimentos para estimar parâmetros
- Avaliar robustez do modelo
- Simplificar modelos complexos

Coeficiente de Sensibilidade

$$S_{x,p} = \frac{\partial x}{\partial p} \cdot \frac{p}{x} = \frac{\frac{\partial \ln}{\partial x}}{\frac{\partial \ln}{\partial p}}$$

Sensibilidade relativa normalizada

Tipos de Análise de Sensibilidade I

Local

- Variação em torno de um ponto nominal
- Derivadas parciais
- Rápida e analítica (quando possível)

Global

- Exploração de todo o espaço de parâmetros
- Métodos de amostragem (Monte Carlo, Sobol)
- Computacionalmente intensiva

Tipos de Análise de Sensibilidade II

Trade-off

Local: rápida mas limitada

Global: abrangente mas custosa

Analise de Sensibilidade Local I

```
1 def sensitivity_local(model, params, param_names,
2                       perturbation=0.01):
3     """
4     Analise de sensibilidade local
5     """
6     baseline = model(params)
7     sensitivities = {}
8     for i, name in enumerate(param_names):
9         params_pert = params.copy()
10        params_pert[i] *= (1 + perturbation)
11        output_pert = model(params_pert)
12        sensitivities[name] = (output_pert - baseline) / baseline
13        / perturbation
14    return sensitivities
```

Analise de Sensibilidade Local II

```
15 # Exemplo
16 sens = sensitivity_local(lambda p: p[1], [0.5, 100, 10], ['r', 'K', 'NO'])
```

Listing 12: Sensibilidade por diferenças finitas

Estimação de Parâmetros I

Problema

Dados: $(t_1, y_1), (t_2, y_2), \dots, (t_n, y_n)$

Modelo: $\hat{y}(t; \mathbf{p})$

Objetivo: Encontrar $\mathbf{p}^*$ que melhor ajusta os dados

Função Objetivo

Mínimos quadrados:

$$\text{SSE}(\mathbf{p}) = \sum_{i=1}^n [y_i - \hat{y}(t_i; \mathbf{p})]^2$$

Objetivo: $\min_{\mathbf{p}} \text{SSE}(\mathbf{p})$

Métodos de Otimização

- Levenberg-Marquardt
- Evolução Diferencial
- Algoritmos Genéticos

Ajuste de Curvas com scipy I

```
1 from scipy.optimize import curve_fit
2
3 # Dados experimentais
4 t_data = np.linspace(0, 10, 20)
5 N_data = 100/(1+9*np.exp(-0.5*t_data)) + np.random.normal(0, 2,
6     20)
7
8 # Modelo e Ajuste
9 def logistic_model(t, r, K): return K / (1 + (K/10 - 1)*np.exp(-r
10     *t))
11 p_opt, _ = curve_fit(logistic_model, t_data, N_data, p0=[1.0,
12     50])
13
14 # Plotar
15 plt.plot(t_data, N_data, 'o', label='Dados')
```

Ajuste de Curvas com scipy II

```
13 plt.plot(t_data, logistic_model(t_data, *p_opt), '-', label='  
    Ajuste')
```

Listing 13: Estimação de parâmetros

Métricas de Qualidade do Ajuste

- **SSE:** Soma dos erros quadrados
- **R²:** Coeficiente de determinação (0 a 1)
- **RMSE:** Raiz do erro quadrático médio
- **AIC/BIC:** Critérios de informação (penalizam complexidade)

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Atenção!

- R^2 alto não garante bom modelo
- Validar com dados independentes
- Verificar plausibilidade biológica

Estudo de Caso: Dinâmica de Infecção Viral

Sistema

Modelo SIR (Susceptible-Infected-Recovered):

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

Número Básico de Reprodução (R_0)

$$R_0 = \frac{\beta}{\gamma}$$

Parâmetros do Modelo SIR

onde:

- S = susceptíveis
- I = infectados
- R = recuperados
- β = taxa de transmissão
- γ = taxa de recuperação
- $N = S + I + R$ = população total

Definição: R_0 (R-naught)

Numero médio de infecções secundárias causadas por um indivíduo infectado em uma população totalmente susceptível.

$$R_0 = \frac{\beta}{\gamma}$$

Numero Básico de Reprodução II

Interpretação

- $R_0 < 1$: epidemia se extingue
- $R_0 > 1$: epidemia se propaga
- $R_0 = 1$: limiar epidêmico

Exemplo: COVID-19

R_0 estimado entre 2-6, dependendo da variante e medidas de controle

Implementação do Modelo SIR I

```
1 def sir_model(y, t, beta, gamma):  
2     S, I, R = y  
3     N = S + I + R  
4     return [-beta * S * I / N, beta * S * I / N - gamma * I,  
5             gamma * I]  
6  
7 # Parametros e condicoes iniciais  
8 beta, gamma = 0.5, 0.1  
9 R0 = beta / gamma  
10 N, I0 = 1000, 10  
11 y0 = [N - I0, I0, 0]
```

Listing 14: Modelo SIR completo

```
1 S, I, R = sol.T
2 plt.figure(figsize=(10, 6))
3 plt.plot(t, S, 'b-', label='Suscept.'), plt.plot(t, I, 'r-',
4           label='Infect.')
5 plt.plot(t, R, 'g-', label='Recuper.')
6
7 # Estatísticas
8 peak_idx = np.argmax(I)
9 print(f"Pico: {I[peak_idx]:.0f} em {t[peak_idx]:.1f} dias")
10 print(f"Ataque final: {R[-1]/N*100:.1f}%")
```

Listing 15: Análise do modelo SIR

Por que Modelos Estocásticos?

Limitações dos Modelos Determinísticos

- Assumem grande número de moléculas e variáveis contínuas
- Ignoram flutuações térmicas e variabilidade molecular
- Inadequados quando $N \lesssim 100$ moléculas ou eventos raros

Quando usar modelos estocásticos?

- Movimentos de moléculas individuais
- Sistemas com ruído térmico relevante (escala nanométrica)
- Polímeros em confinamento (gel, cromatina, poros)
- Expressão gênica com poucos fatores de transcrição

Importante!

Flutuações não são "ruído a eliminar"— frequentemente carregam informação biológica.

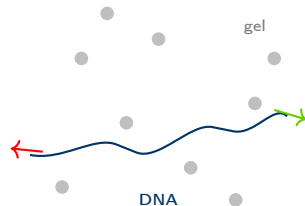
Movimento de Polímeros: O Modelo de Reptation

Definição: Reptation

Mecanismo de movimento de um polímero longo (como o DNA) confinado em uma rede ou gel, proposto por P.G. de Gennes (1971). O polímero se move como uma cobra dentro de um tubo virtual formado pelos obstáculos ao seu redor.

Ideia central:

- O DNA não pode atravessar os obstáculos da rede
- Cria um “tubo” ao longo de seu contorno
- Move-se pela criação/destruição de alças nas extremidades
- Movimento resultante: difusão ao longo do próprio eixo



Formulação Estocástica do Reptation (1/2)

Modelo de Caminhada Aleatória no Tubo

O DNA é representado por N segmentos de comprimento b . A cada passo de tempo Δt , cada extremidade:

- Com prob. $\frac{1}{2}$: **avança** — adiciona um segmento à frente
- Com prob. $\frac{1}{2}$: **recua** — remove um segmento da frente

Coefficiente de difusão curvilinear:

$$D_c = \frac{b^2}{2\tau}$$

Formulação Estocástica do Reptation (2/2)

Coeficiente de difusão efetivo (3D):

$$D \propto \frac{D_c}{N^2} \sim \frac{1}{N^2}$$

Predição chave

O tempo de renovação do tubo escala como $\tau_{\text{rep}} \sim N^3$ — verificado experimentalmente por eletroforese em gel.

Importante!

Quanto mais longo o DNA, mais lento o movimento — base da separação por tamanho em gel.

Reptation em Campo Elétrico (Eletroforese)

Biased Reptation (Duke, 1989)

Na presença de campo elétrico E , a probabilidade de avanço/recuo nas extremidades se torna assimétrica:

$$p_+ = \frac{1}{2} \left(1 + \frac{qEb}{k_B T} \right) \quad (\text{avança})$$
$$p_- = \frac{1}{2} \left(1 - \frac{qEb}{k_B T} \right) \quad (\text{recua})$$

Mobilidade eletroforética:

$$\mu = \frac{v}{E} \propto \frac{1}{N}$$

- Fragmentos menores: maior mobilidade
- Base da separação por gel eletroforese
- Válido para DNA **longo** (regime de reptation)

Limite do modelo

Para campos altos ($qEb \gg k_B T$): DNA se alinha com o campo e a separação por tamanho se perde.

Implementação: Simulação de Reptation I

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def simular_reptation(N, n_steps, E=0.0, kBT=1.0, q=1.0, b=1.0):
5     """
6     Simula reptation de DNA em campo eletrico E.
7     N          : numero de segmentos
8     n_steps    : passos de Monte Carlo
9     E          : campo eletrico (0 = difusao pura)
10    """
11    # Posicao do centro de massa ao longo do tubo
12    pos_cm = np.zeros(n_steps)
13    pos = 0.0 # deslocamento acumulado
14
15    # Probabilidade de avanco
```

Implementação: Simulação de Reptation II

```
16     p_mais = 0.5 * (1.0 + q * E * b / kBT)
17     p_mais = np.clip(p_mais, 0.0, 1.0)
18
19     for t in range(1, n_steps):
20         # Sortear qual extremidade se move (frente ou tras)
21         r = np.random.rand()
22         if r < p_mais:
23             pos += b / N      # deslocamento do CM: b/N por passo
24         else:
25             pos -= b / N
26         pos_cm[t] = pos
27
28     return pos_cm
```

Listing 16: Simulação estocástica de reptation

Implementação: Análise e Visualização

```
1 n_steps = 50000
2 tamanhos = [50, 100, 200]    # numero de segmentos
3 E = 0.1                       # campo eletrico fraco
4
5 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
6
7 for N in tamanhos:
8     traj = simular_reptation(N, n_steps, E=E)
9     t = np.arange(n_steps)
10
11     # Trajetoria
12     axes[0].plot(t, traj, label=f'N={N}', alpha=0.7)
13
14     # MSD (mean squared displacement)
15     msd = np.cumsum((traj - traj[0])**2) / np.arange(1, n_steps+1)
16     axes[1].loglog(t[1:], msd[1:], label=f'N={N}')
17
18 axes[0].set(xlabel='Passos', ylabel='Posicao CM', title='Trajetorias')
19 axes[1].set(xlabel='Passos', ylabel='MSD', title='MSD (escala log-log)')
20
21 for ax in axes:
22     ax.legend(); ax.grid(True)
23 plt.tight_layout()
```

Listing 17: Comparacao entre tamanhos de DNA

Predições e Verificação Experimental

Escalonamentos do Modelo de Reptation

Grandeza	Predição	Verificação
Difusividade	$D \sim N^{-2}$	Eletroforese em campo nulo
Mobilidade	$\mu \sim N^{-1}$	Gel de agarose, campo baixo
Tempo de renovação	$\tau \sim N^3$	Birrefringência transitória
MSD curvilinear	$\langle s^2 \rangle \sim t$	FCS, tracking de moléculas

Aplicações Biotecnológicas

- **Eletroforese em gel de agarose:** separação de fragmentos de DNA por tamanho
- **Sequenciamento de DNA:** entender mobilidade de fragmentos
- **Eletroforese em campo pulsado (PFGE):** separação de cromossomos inteiros
- **Nanoporos:** translocação de DNA — extensão do modelo de reptation

Princípios de Modelagem

1. Simplicidade vs Realismo

- Comece simples, adicione complexidade gradualmente
- "Make things as simple as possible, but not simpler-- Einstein
- Balanceie entre descrição acurada e tratabilidade

2. Validação

- Compare com dados experimentais
- Teste previsões em novos experimentos
- Valide em múltiplas escalas/condições

3. Documentação

- Documente hipóteses claramente
- Registre origem de parâmetros
- Mantenha código reproduzível

Erros Frequentes

1. **Overfitting:** modelo complexo demais para os dados
2. **Parâmetros não-identificáveis:** múltiplas soluções
3. **Unidades inconsistentes:** sempre verifique dimensões
4. **Condições iniciais irrealistas:** valores negativos, etc.
5. **Extrapolação excessiva:** além do regime validado
6. **Ignorar incerteza:** sempre quantifique incerteza

Importante!

"Remember that all models are wrong; the practical question is how wrong do they have to be to not be useful— George Box

Ferramentas Computacionais

Python

- NumPy, SciPy
- Matplotlib, Seaborn
- PyDSTool
- Tellurium
- COBRApy (metabolismo)

R

- deSolve
- FME (fitting)
- ggplot2

MATLAB

- ode45, ode15s
- SimBiology
- Simbólico

Especializadas

- COPASI
- CellDesigner
- Virtual Cell
- BioNetGen

Questão 1: Modelo de Crescimento

Considere um modelo de crescimento com colheita constante:

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right) - H$$

1. Encontre os pontos de equilíbrio
2. Analise a estabilidade para $H < rK/4$ e $H > rK/4$

Questão 2: Método Numérico

Implemente o método de Euler para resolver:

$$\frac{dy}{dt} = -y + \sin(t), \quad y(0) = 1$$

Compare com a solução analítica para diferentes passos.

Questão 3: Sistema Acoplado

Considere o sistema de competição:

$$\begin{aligned}\frac{dN_1}{dt} &= r_1 N_1 \left(1 - \frac{N_1 + \alpha N_2}{K_1} \right) \\ \frac{dN_2}{dt} &= r_2 N_2 \left(1 - \frac{N_2 + \beta N_1}{K_2} \right)\end{aligned}$$

1. Implemente o modelo em Python
2. Encontre os pontos de equilíbrio
3. Simule para diferentes valores de α e β
4. O que acontece quando $\alpha\beta > 1$ vs $\alpha\beta < 1$?

Exercícios - Parte 3

Projeto: Modelo SIR com Vacinação

Estenda o modelo SIR para incluir vacinação:

$$\frac{dS}{dt} = -\beta \frac{SI}{N} - \nu S, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I + \nu S$$

1. Implemente o modelo e calcule o R_0 efetivo
2. Determine ν necessária para controle ($R_0^{eff} < 1$)
3. Simule diferentes estratégias de vacinação

Referências Principais

-  Strogatz S.H. (2018). *Nonlinear Dynamics and Chaos*, 2nd ed. CRC Press.
-  Edelstein-Keshet L. (2005). *Mathematical Models in Biology*. SIAM.
-  Britton N.F. (2003). *Essential Mathematical Biology*. Springer.
-  Murray J.D. (2002). *Mathematical Biology I: An Introduction*, 3rd ed. Springer.
-  Keener J., Sneyd J. (2009). *Mathematical Physiology*, 2nd ed. Springer.

Livros Avançados

- Ellner S.P., Guckenheimer J. (2011). *Dynamic Models in Biology*
- Alon U. (2019). *An Introduction to Systems Biology*, 2nd ed.
- Klipp E. et al. (2016). *Systems Biology*, 2nd ed.

Recursos Online

- **Wolfram MathWorld:** <https://mathworld.wolfram.com>
- **MIT OCW:** Mathematical Biology course
- **Scipy Lecture Notes:** <https://scipy-lectures.org>

Tutoriais Python

- <https://scipy-cookbook.readthedocs.io>
- <https://python-numerics.github.io>

Obrigado!

Capítulo 2: Introdução à Modelagem Matemática

Próximo Capítulo:
Static Network Models

Dúvidas? Perguntas?